

Drakens Energy 360

*A synthetic end-to-end data engineering platform
for a downstream energy business*

37.2m

FACT ROWS

85

FACT TABLES

70

CONFORMED DIMENSIONS

63m

DEFECTS INJECTED

587

SITES

26

AFRICAN MARKETS

250

OF THEM IN SA

9

SA PROVINCES

Databricks · Delta Lake · Unity Catalog · dbt · Azure · MLflow · Power BI
2026-09-09

Disclaimer. Independent synthetic portfolio project. This document and the platform it describes contain no data from any real company. Drakens Energy is a fictional company. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident is randomly generated. Company and person names are invented; any resemblance to a real organisation or individual is coincidental and unintended. Site coordinates are town centroids plus random jitter and do not represent any real service station.

Contents

1. The brief
2. What was built
3. Data worth modelling on
4. Then it had to be broken
5. Cleansing that accounts for everything
6. Two defects it found in itself
7. Architecture
8. The data model
9. Decision support
10. Machine learning
11. Governance
12. Running on Databricks
13. What I would do differently

1. The brief

Build a data platform for a downstream energy business: a retail forecourt network, a commercial B2B book, primary distribution, LPG, lubricants, aviation, marine, loyalty and digital channels, EV and solar, asset maintenance, safety, and finance. Model it on South Africa. Make it real enough that the decisions taken on it would be defensible.

The additional constraint I set myself: **the data has to be dirty**. A platform demonstrated on clean data demonstrates almost nothing, because the difficult part of data engineering is not the join. It is deciding what to do with the row where `litres` reads `"1 234,5"`, the site code has a trailing space, and the same transaction arrived twice because the point-of-sale replayed its batch.

2. What was built

37.2m

FACT ROWS

85

FACT TABLES

70

CONFORMED DIMENSIONS

63m

DEFECTS INJECTED

587

SITES

26

AFRICAN MARKETS

250

OF THEM IN SA

9

SA PROVINCES

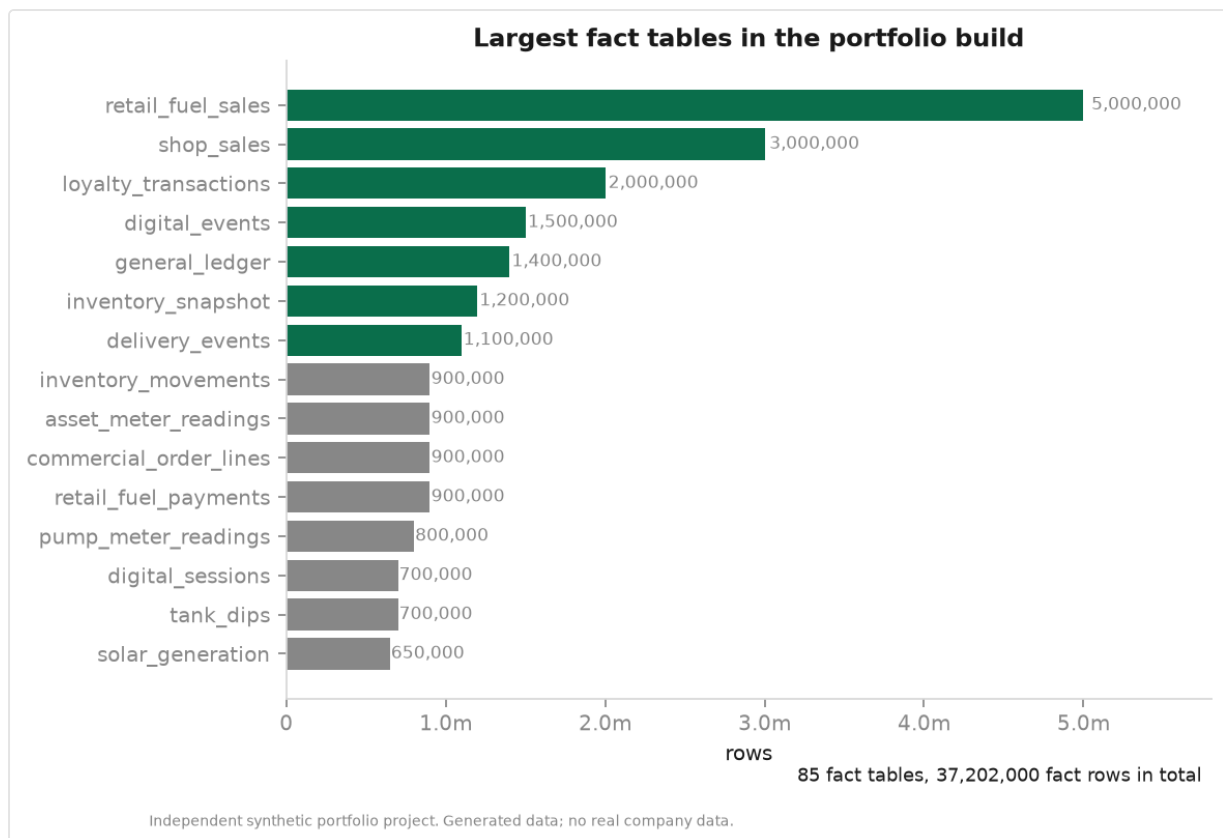


Figure 1. The largest fact tables produced by the portfolio build.

3. Data worth modelling on

Random numbers are not a dataset. If transaction timestamps are uniform, a demand forecast has nothing to learn and any accuracy figure it produces is meaningless. The generator therefore encodes the actual rhythms of the business: commute peaks, a Friday-heavy week, December holiday travel, LPG demand rising through winter, and the fact that the day *before* a public holiday is the strongest fuel day of the year.

South African pump prices are regulated and adjusted monthly, so the synthetic price is a per-month step function rather than per-transaction noise. Each site carries a `demand_index` combining urban class, national-route position and offer mix, so a Gauteng truck stop and a rural single-product site behave differently.

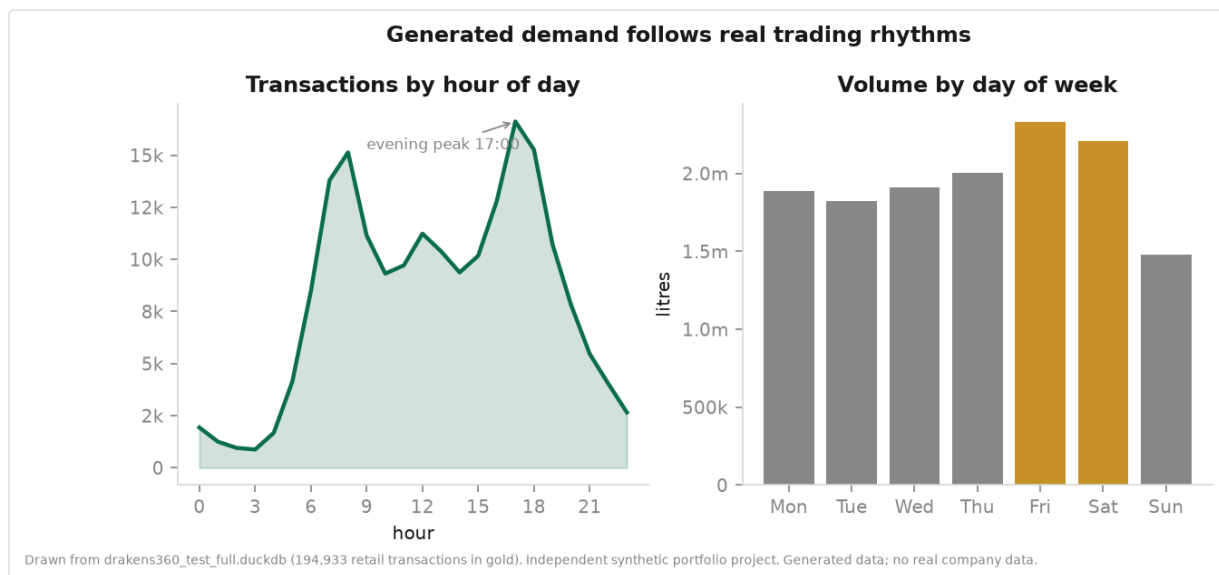


Figure 2. Generated demand follows real trading rhythms: commute peaks at 08:00 and 17:00, and a Friday-heavy week. Without this the forecasting model would have nothing to learn.

This took two attempts. The first version weighted which assets generated maintenance work orders but drew the work type independently, so the breakdown rate came out flat across asset age — 12%, 16%, 12%, 11% — and the predictive-maintenance model had nothing to find. After making breakdown probability depend on the asset's own condition, the rate rises 23% → 39% → 51% → 59% across age bands. Same row count; the difference is whether the dataset means anything.

4. Then it had to be broken

62,924,526 defects were injected across the 37,202,000-row build, modelled on the specific systems being simulated rather than sprinkled at random.

Defects injected into the landing zone.

DEFECT CLASS	COUNT
numeric delivered as text	56,600,000
locale decimal comma	1,697,958
whitespace padding	1,504,943
thousands separator	1,097,944
case inconsistency	738,361
sentinel string	474,245
numeric sentinel	207,191
null required field	187,192
duplicate rows	93,468
negative measure	86,613
truncated identifier	71,093
orphan foreign key	47,436
unflagged currency	41,590
epoch timestamp	31,612
future timestamp	23,742
derived value mismatch	12,256
unit of measure error	7,233
encoding mojibake	1,649

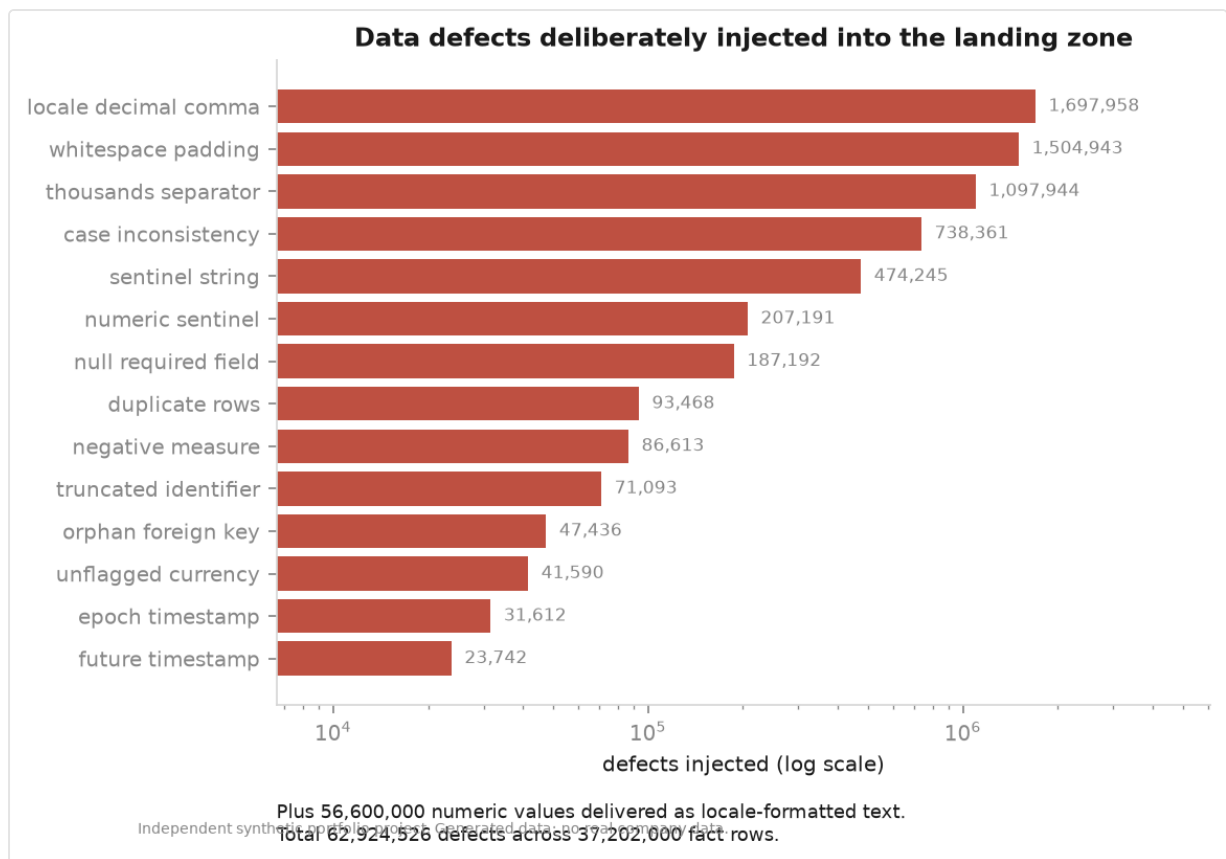


Figure 3. Defects deliberately injected into the landing zone, by class. Every one is counted at injection time, which is what makes the cleansing layer measurable rather than merely asserted.

Every defect is counted as it is injected and written to the build manifest. That is the design decision that matters: it converts cleansing from a claim into a measurement.

5. Cleansing that accounts for everything

167 generated staging models repair, validate and deduplicate. 122 quarantine models keep every rejected row alongside the rule that rejected it. The reconciliation that has to hold is simple:

```
raw rows = cleansed + quarantined + duplicates removed
```

A row that is neither cleaned nor rejected nor identified as a duplicate has been silently lost, and a silently lost row becomes an unexplainable variance three months later that nobody can trace.

Cleansing outcome per monitored feed.

TABLE NAME	RAW ROWS	CLEANSED ROWS	QUARANTINED ROWS	DUPLICATES REMOVED	QUARANTINE RATE PCT
fact_retail_fuel_sales	200,000	194,933	4,572	495	2.29
fact_shop_sales	120,000	116,702	2,992	306	2.49
fact_general_ledger	84,000	83,256	540	204	0.64
fact_loyalty_transactions	80,000	78,944	857	199	1.07
fact_delivery_events	66,000	65,557	283	160	0.43
fact_digital_events	60,000	59,593	236	171	0.39
fact_retail_fuel_payments	54,000	53,382	489	129	0.91
fact_commercial_order_lines	54,000	53,145	728	127	1.35
fact_asset_meter_readings	54,000	53,453	397	150	0.73
fact_inventory_movements	54,000	53,201	656	143	1.22
fact_inventory_snapshot	50,000	49,300	583	117	1.17
fact_pump_meter_readings	48,000	47,383	502	115	1.05
fact_tank_dips	42,000	41,351	543	106	1.29
fact_digital_sessions	42,000	41,745	162	93	0.39
fact_customer_invoices	37,200	36,733	374	93	1.00
fact_site_energy_consumption	36,000	35,623	305	72	0.85
fact_customer_receipts	33,600	33,205	301	94	0.90
fact_mobile_payments	31,200	30,842	284	74	0.91
fact_site_daily_operations	30,000	29,556	365	79	1.22
dim_loyalty_member	30,000	30,000	0	0	0.00
fact_loyalty_points_balance	28,800	28,491	232	77	0.81
fact_solar_generation	26,000	25,704	228	68	0.88
fact_finance_revenue_cost	25,200	22,476	2,662	62	10.56
fact_forecourt_shift	25,200	24,915	217	68	0.86
dim_digital_user	25,000	25,000	0	0	0.00
fact_commercial_orders	25,000	24,417	528	55	2.11
fact_ev_charger_status	24,000	23,737	199	64	0.83
fact_grid_import_export	22,800	22,586	152	62	0.67
fact_commercial_deliveries	22,800	22,447	293	60	1.28
fact_accounts_receivable	22,800	22,508	232	60	1.02

TABLE NAME	RAW ROWS	CLEANSED ROWS	QUARANTINED ROWS	DUPLICATES REMOVED	QUARANTINE RATE PCT
fact_daily_executive_kpi	22,800	22,385	357	58	1.57
fact_vehicle_trips	20,400	20,247	100	53	0.49
fact_opex	19,200	18,995	164	41	0.85
fact_customer_credit_exposure	18,000	17,735	234	31	1.30
fact_accounts_payable	18,000	17,848	102	50	0.57
fact_forecast	18,000	17,739	222	39	1.23
fact_lpg_cylinder_movements	18,000	17,814	143	43	0.79
fact_lpg_sales	16,000	15,627	331	42	2.07
fact_price_changes	15,600	15,353	204	43	1.31
fact_budget	15,600	15,479	87	34	0.56
fact_replenishment_orders	15,600	15,362	192	46	1.23
fact_campaign_responses	15,600	15,470	92	38	0.59
fact_training_compliance	15,600	15,442	123	35	0.79
fact_deliveries	15,000	14,796	175	29	1.17
fact_budget_vs_actual	14,400	14,262	102	36	0.71
fact_lubricant_sales	14,000	13,687	281	32	2.01
fact_terminal_issues	12,600	12,463	95	42	0.75
fact_asset_inspections	12,000	11,879	93	28	0.78
fact_spare_parts_usage	11,400	11,292	83	25	0.73
fact_procurement_orders	11,400	11,268	110	22	0.96
fact_safety_observations	10,800	10,715	55	30	0.51
fact_fleet_costs	10,800	10,704	59	37	0.55
fact_terminal_throughput	10,800	10,686	93	21	0.86
fact_procurement_receipts	10,000	9,858	109	33	1.09
fact_energy_costs	9,600	9,470	111	19	1.16
fact_lubricant_orders	9,000	8,882	94	24	1.04
fact_route_performance	9,000	8,926	56	18	0.62
fact_permit_to_work	9,000	8,910	73	17	0.81
bridge_customer_contract	8,947	6,000	0	2,947	0.00
fact_terminal_receipts	8,400	8,314	64	22	0.76
fact_aircraft_uplifts	7,800	7,729	49	22	0.63

TABLE NAME	RAW ROWS	CLEANSED ROWS	QUARANTINED ROWS	DUPLICATES REMOVED	QUARANTINE RATE PCT
fact_lpg_inventory_snapshot	7,200	7,126	60	14	0.83
fact_contract_pricing	7,200	7,123	62	15	0.86
fact_cashflow	7,200	7,125	61	14	0.85
fact_asset_downtime	7,200	7,117	69	14	0.96
fact_inventory_adjustments	7,200	7,110	74	16	1.03
fact_lubricant_inventory_snapshot	6,600	6,513	75	12	1.14
dim_customer	6,000	5,852	148	0	2.47
bridge_customer_hierarchy	6,000	300	0	5,700	0.00
fact_ev_charging_sessions	6,000	5,886	102	12	1.70
fact_working_capital	5,400	5,337	44	19	0.81
fact_vehicle_safety_events	5,400	5,352	34	14	0.63
fact_contract_volume_commitments	5,400	5,337	50	13	0.93
fact_aviation_sales	5,000	4,897	88	15	1.76
dim_asset	5,000	5,000	0	0	0.00
bridge_asset_site	5,000	5,000	0	0	0.00
bridge_employee_site	4,256	3,000	0	1,256	0.00
fact_customer_service_cases	4,200	4,154	38	8	0.91
fact_asset_failures	4,200	4,142	46	12	1.09
fact_lpg_bulk_deliveries	4,200	4,155	33	12	0.79
fact_marine_sales	4,000	3,921	71	8	1.77
dim_pump	4,000	4,000	0	0	0.00
fact_maintenance_work_orders	4,000	3,958	38	4	0.95
fact_stock_losses	3,600	3,550	35	15	0.97
fact_shop_returns	3,600	3,548	41	11	1.14
fact_bunker_deliveries	3,600	3,571	18	11	0.50
bridge_site_product	3,536	600	0	2,936	0.00
dim_contract	3,500	3,500	0	0	0.00
bridge_customer_site	3,172	1,500	0	1,672	0.00
dim_employee	3,000	3,000	0	0	0.00
fact_near_misses	2,760	2,729	24	7	0.87
dim_tank	2,500	2,500	0	0	0.00

TABLE NAME	RAW ROWS	CLEANSED ROWS	QUARANTINED ROWS	DUPLICATES REMOVED	QUARANTINE RATE PCT
fact_capex	2,400	2,369	26	5	1.08
fact_promotions	2,400	397	223	1,780	9.29
fact_aviation_inventory_snapshot	2,400	2,365	27	8	1.12
bridge_route_site	1,869	600	0	1,269	0.00
fact_marine_inventory_snapshot	1,800	1,776	16	8	0.89
fact_environmental_events	1,320	1,296	18	6	1.36
dim_equipment	1,250	1,250	0	0	0.00
bridge_supplier_product	1,206	400	0	806	0.00
dim_driver	1,100	1,100	0	0	0.00
bridge_site_promotion	1,018	600	0	418	0.00
dim_vehicle	900	900	0	0	0.00
fact_hsseq_incidents	800	795	4	1	0.50
bridge_site_hierarchy	600	586	14	0	2.33
dim_route	600	600	0	0	0.00
dim_site	600	586	14	0	2.33
fact_import_cargoes	540	538	2	0	0.37
dim_ev_charger	400	400	0	0	0.00
dim_supplier	400	385	15	0	3.75
dim_solar_asset	300	300	0	0	0.00
dim_city	88	88	0	0	0.00
dim_terminal	60	60	0	0	0.00
dim_promotion	44	44	0	0	0.00
dim_depot	40	40	0	0	0.00
dim_campaign	38	38	0	0	0.00
dim_product	31	31	0	0	0.00
bridge_product_hierarchy	31	31	0	0	0.00
dim_price_zone	30	30	0	0	0.00
dim_country	29	28	1	0	3.45
dim_currency	25	25	0	0	0.00
dim_carrier	22	22	0	0	0.00
dim_warehouse	20	20	0	0	0.00

TABLE NAME	RAW ROWS	CLEANSED ROWS	QUARANTINED ROWS	DUPLICATES REMOVED	QUARANTINE RATE PCT
dim_airport	12	12	0	0	0.00
dim_legal_entity	12	12	0	0	0.00
dim_province	9	9	0	0	0.00
dim_port	8	8	0	0	0.00
dim_lpg_cylinder_type	4	4	0	0	0.00

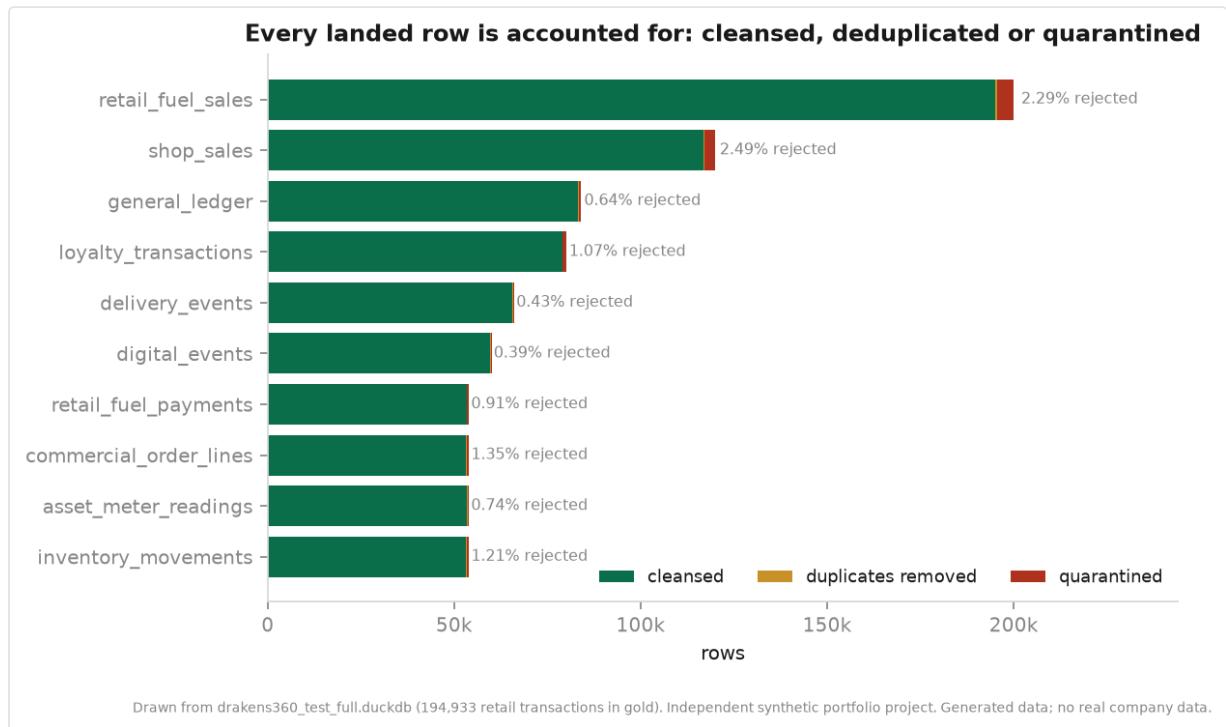


Figure 4. Every landed row is accounted for as cleansed, deduplicated or quarantined. A row that is none of the three has been silently lost.

The hardest single rule

Numeric parsing. A comma is ambiguous: in "1,234.56" it is a thousands separator, in "1234,56" it is the decimal mark. Getting this wrong does not throw an error — it produces a number wrong by three orders of magnitude that looks entirely plausible. The rule is positional, documented once, and applied identically in all 167 models, because the same value parsed two different ways in two tables is worse than not parsing it at all.

Top rejection reasons, with the team likely to own the fix.

FAILED RULE	FAILURE CATEGORY	LIKELY OWNER	FAILURES
amounts_reconcile	Other	Data Platform	2,873.00
product_id_present	Missing Required Field	Source System	1,731.00
site_id_present	Missing Required Field	Source System	1,699.00
sales_match_quantity_and_price	Other	Data Platform	1,452.00
gross_margin_zar_non_negative	Invalid Sign	Business Process	1,166.00
unit_price_zar_plausible	Other	Data Platform	660.00
transaction_id_present	Missing Required Field	Source System	630.00
event_timestamp_valid	Invalid Timestamp	Source System	506.00
gross_sales_zar_non_negative	Invalid Sign	Business Process	473.00
unit_price_zar_non_negative	Invalid Sign	Business Process	460.00

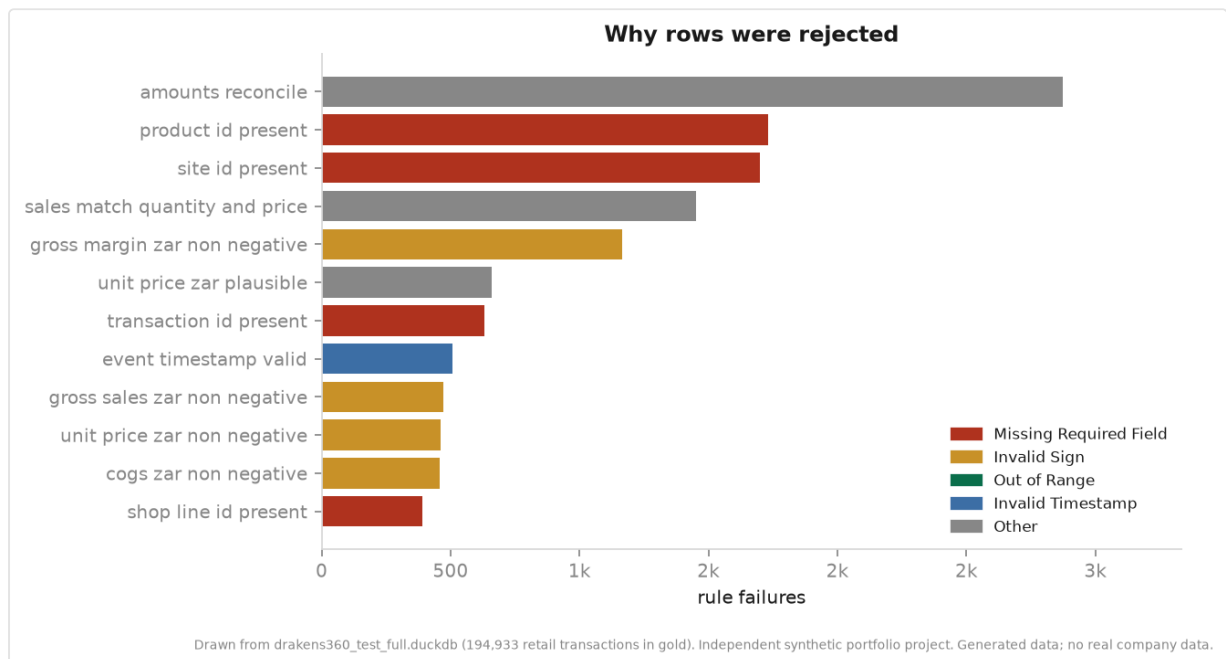


Figure 5. Why rows were rejected. The operational response differs by rule: a missing site code means a broken upstream join, a negative amount usually means credit notes on the wrong feed.

6. Two defects it found in itself

Both were caught by the platform's own controls, which is the entire reason for having them.

Amacro that silently corrupted every ratio

`safe_divide('a - b', 'c')` compiled to `a - (b / c)` because the macro did not parenthesise its arguments. Every margin percentage, utilisation rate and share-of-total in the project was wrong

— not obviously wrong, just quietly incorrect. It was caught by a plausibility test asserting that product margin falls between −5 and 100 per cent. A `not_null` test would never have found it.

A general ledger that did not balance

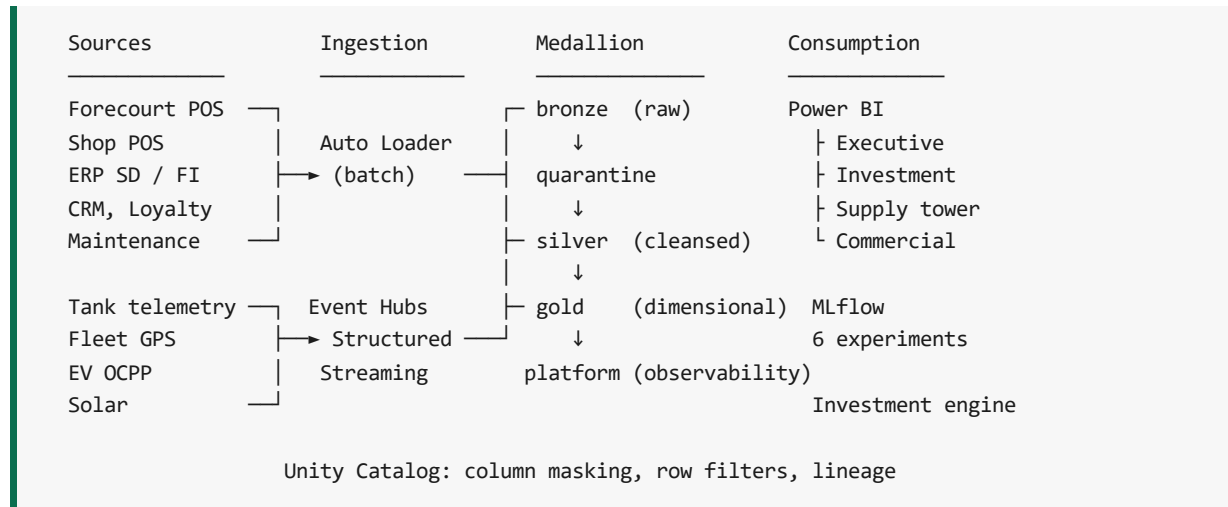
Journal lines were posted independently, so total debits exceeded credits by 46%. No finance function would accept that. It was caught by the reconciliation controls, which evaluate double-entry as *data* rather than only as a test, and fixed by emitting journals as balanced pairs.

Reconciliation controls, evaluated every run.

CONTROL CODE	CONTROL OWNER	VARIANCE PCT	IS PASSING
DELIVERIES_HAVE_KNOWN_DESTINATION	Supply	0.00	pass
GL_DEBITS_EQUAL_CREDITS	Finance	0.00	pass
ORDERS_HAVE_KNOWN_CUSTOMER	Commercial	0.00	pass
RETAIL_MARGIN_RECONCILES	Finance	0.00	pass
RETAIL_VOLUME_TIES_TO_DAILY_AGG	Supply	0.00	pass

The lesson. `not_null` and `unique` catch structural faults. Business-plausibility and reconciliation controls catch the faults that reach a board pack.

7. Architecture



The medallion runs unchanged on DuckDB locally and on Databricks with Delta Lake. Cross-adapter differences are handled in a small set of dbt macros, so the models themselves have no environment-specific branching.

8. The data model

Every diagram in this section is rendered from `model.bim`, the semantic model the Power BI report actually loads. None of them is drawn by hand, and none is transcribed from a design document. That matters more than it sounds: a hand-maintained diagram is correct on the day it is drawn and plausible forever afterwards, and a diagram that quietly disagrees with the warehouse is worse than no diagram at all. These are generated on every build, so a relationship added tomorrow appears in tomorrow's copy, and one removed disappears.

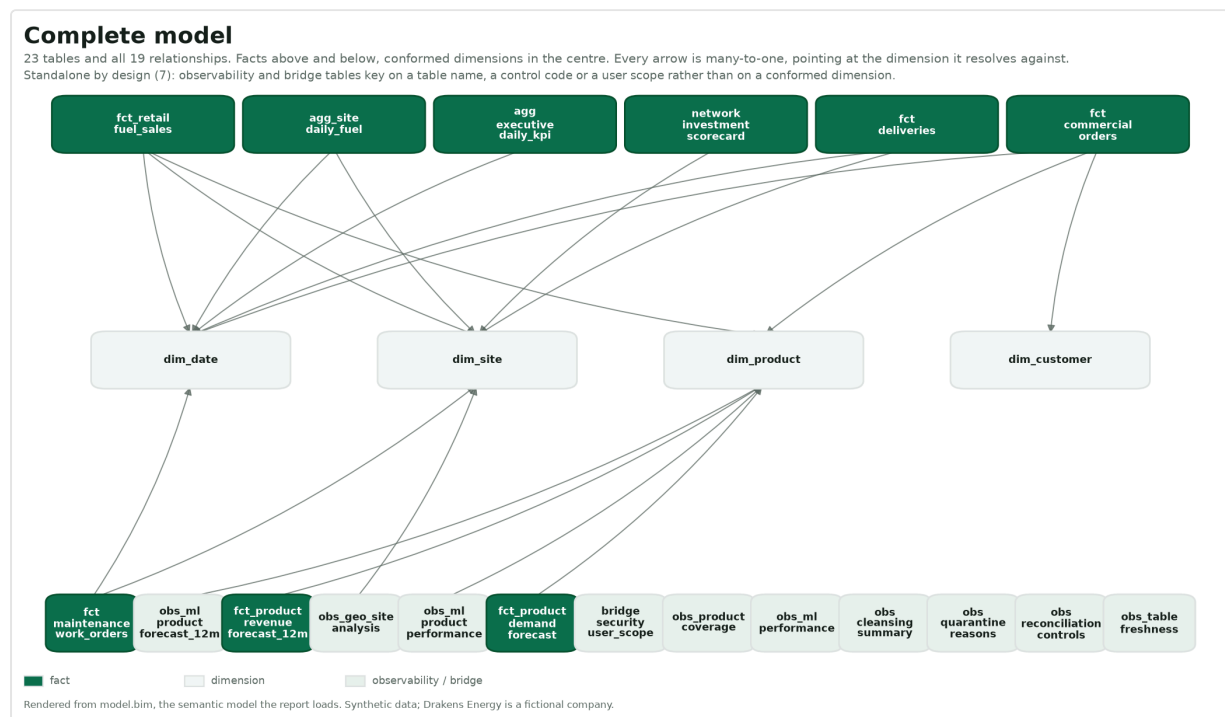


Figure 6. The complete model: 23 tables and every relationship between them. Facts sit above and below a central band of conformed dimensions, and every arrow is many-to-one pointing at the dimension it resolves against. Observability and bridge tables stand alone by design -- they key on a table name, a control code or a user scope rather than on a conformed dimension -- which is a different thing from a fact that has lost its key.

The shape is a set of star schemas sharing their dimensions rather than one schema per subject area. `dim_date` and `dim_site` are the two that carry the network: because a delivery, a maintenance work order and a fuel transaction all resolve against the same site row and the same calendar day, they can be compared without anyone writing a join that reconciles three different notions of “site”.

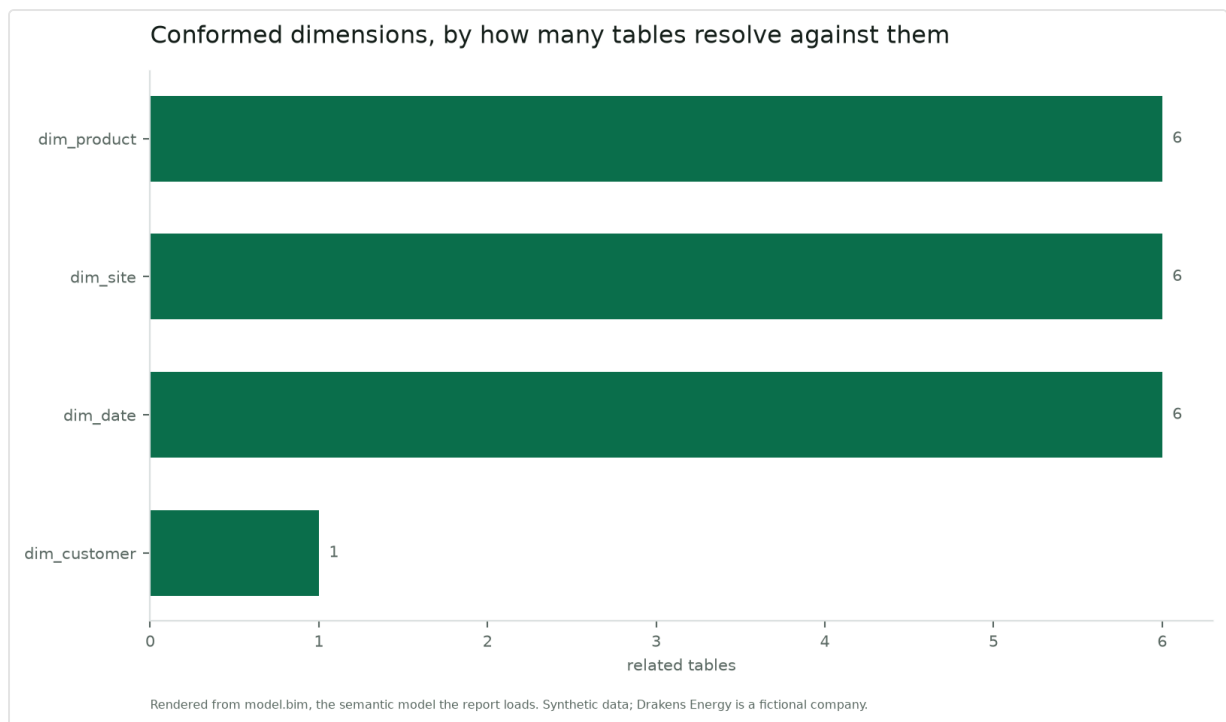
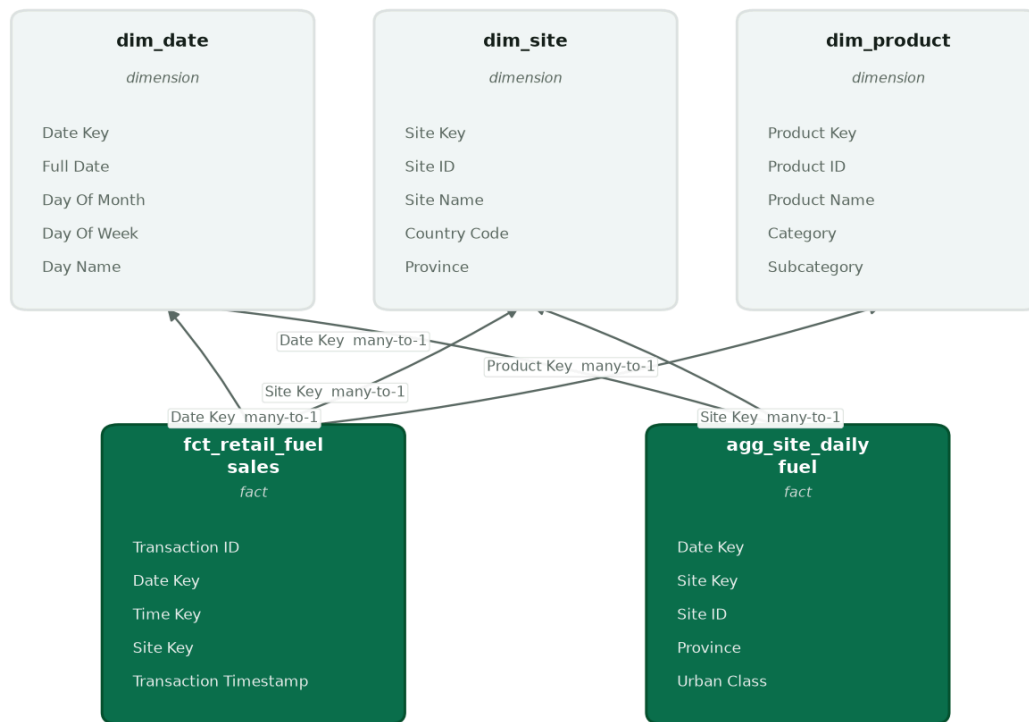


Figure 7. Conformed dimensions ranked by how many tables resolve against them. A dimension only one fact uses is not conformed; it is a lookup table with ambitions. `dim_date` and `dim_site` carry the network, which is what lets a delivery, a work order and a fuel transaction be compared on the same day and the same forecourt.

Two things on these diagrams are absences rather than presences, and both are deliberate. Observability and bridge tables stand alone: they key on a table name, a control code or a user scope, so having no dimension is correct for them. A *fact* with no relationship would be a different matter entirely, and the generator labels that case separately rather than letting the two look alike.

Retail fuel

2 fact table(s), 3 related table(s)



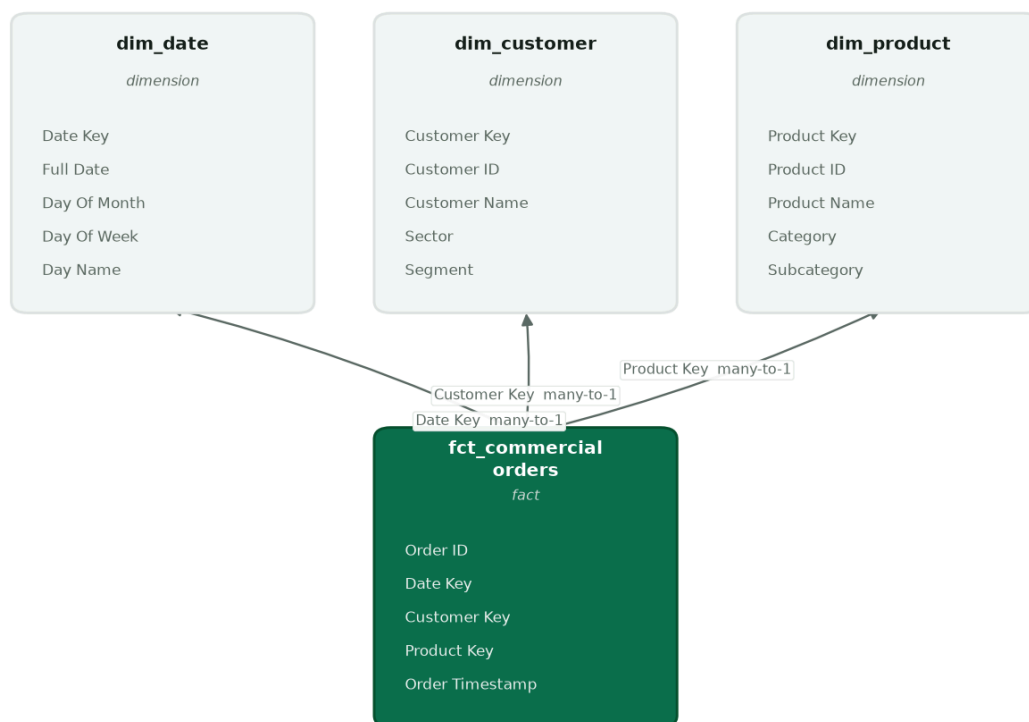
Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Figure 8. Retail fuel in detail. The transaction fact and the site-day aggregate resolve against the same three dimensions, which is what makes the aggregate reconcilable against the detail rather than merely similar to it -- and that reconciliation is one of the five controls that has to pass before a build is accepted.

The retail pair is where the reconciliation lives. Because `fct_retail_fuel_sales` and `agg_site_daily_fuel` resolve against the same three dimensions, the aggregate can be tied back to the transactions row by row — and it is, as one of the five controls that has to pass before a build is accepted. An aggregate built on a different set of keys would still produce a number every morning; it just would not be the same number.

Commercial B2B

1 fact table(s), 3 related table(s)



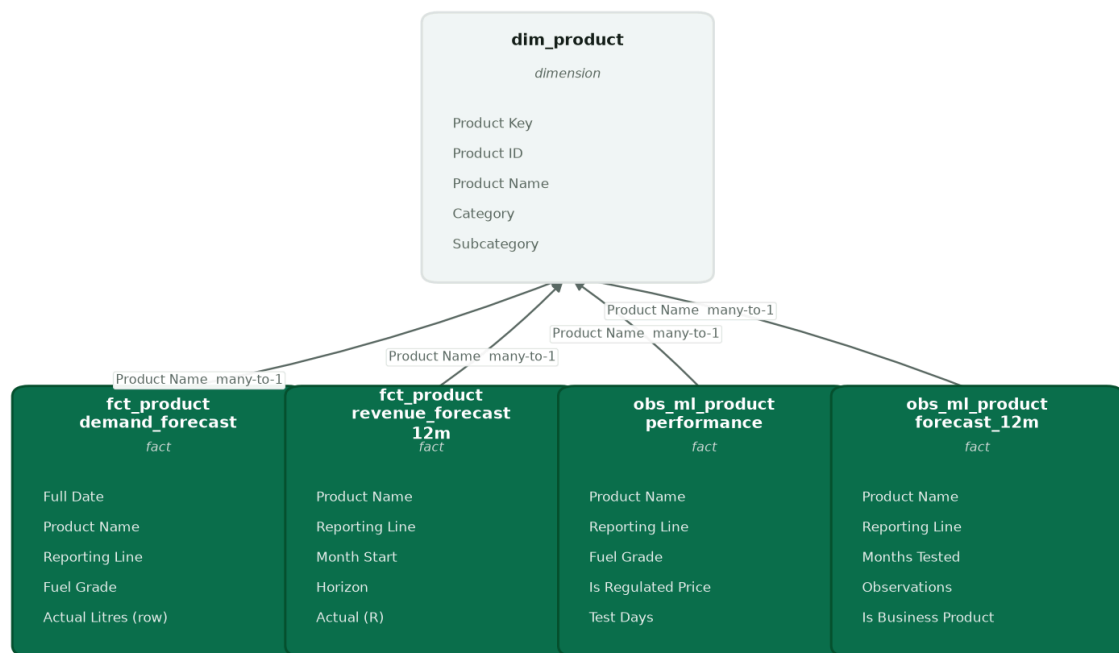
Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Figure 9. Commercial B2B. The relationship from orders to `dim_product` was missing until it was found while writing this: lubricants are R20.3bn of revenue, the largest product line in the business, and no product analysis could see them. Every 'product' figure in the platform was silently a retail-fuel figure.

The commercial diagram records a defect found while writing this chapter. `fct_commercial_orders` carried a `product_key` and had no relationship to `dim_product`, so lubricants — R20.3bn of revenue and the largest product line in the business — could not be sliced by product at all. Every figure the platform called a “product” number was silently a retail-fuel number. The relationship exists now, and the diagram is how it became visible.

Forecasting and model results

4 fact table(s), 1 related table(s)



Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Figure 10. The forecasting tables. These key on product name rather than a surrogate because they are produced by the ML layer rather than by dbt, which is a deliberate seam: the warehouse owns surrogate keys and the model layer is not allowed to mint them.

The forecasting tables key on product name rather than on a surrogate key, which is a deliberate seam rather than an oversight. The warehouse owns surrogate keys; the ML layer is not permitted to mint them, because a model that invents keys quietly becomes a second source of truth for identity. Names are unique across all 32 products, and the relationship is validated on every build like any other.

9. Decision support

`network_investment_scorecard` scores every site by combining trading performance, growth, non-fuel mix, reliability, downtime, safety and asset condition. Percentile ranks put raw figures and incident counts on a common scale, and the weights are an explicit management judgement stated in the SQL rather than a fitted parameter.

Every component is a visible column. A challenged recommendation can be answered with its inputs rather than with “the model said so”, which is the only version of this that survives an executive forum.

Investment recommendation across the network.

INVESTMENT RECOMMENDATION	SITES	AVG SCORE	MARGIN M ZAR
Invest - Expand	15	79.60	2.00
Invest - Maintain Leadership	7	77.80	2.00
Hold	95	62.40	5.00
Invest - Refurbish	181	59.70	11.50
Hold - Strategic Coverage	70	42.50	4.10
Review	208	40.70	7.20
Divest Candidate	11	19.30	0.20

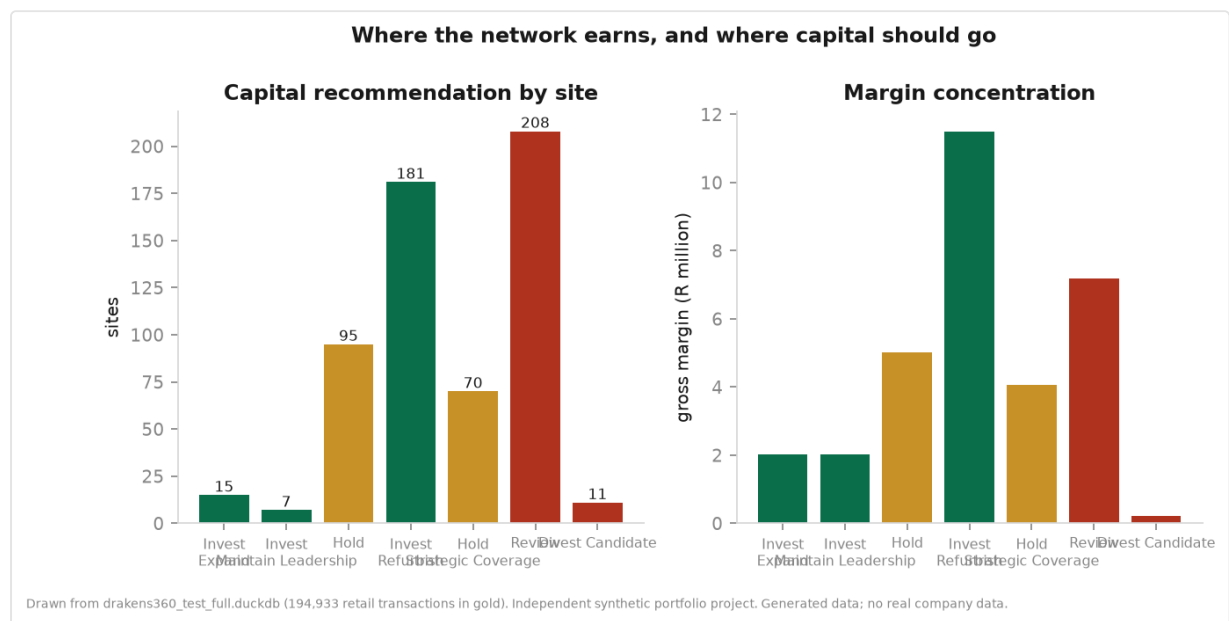


Figure 11. Capital recommendation and margin concentration across the network.

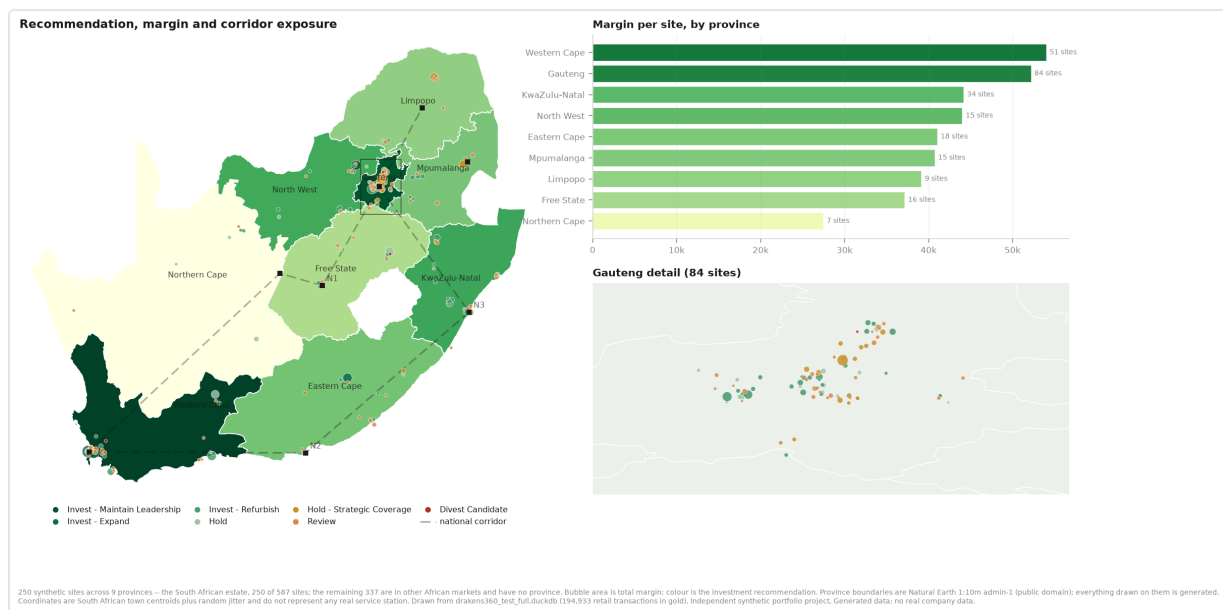


Figure 12. The synthetic South African network. Provinces are shaded by margin per site; bubble area is total margin and colour is the investment recommendation, so a large red bubble -- a high-margin site recommended for divestment -- is visible without a sort. The N1, N2 and N3 are drawn because corridor sites are protected from divestment regardless of score. Shaded areas are the convex hull of each province's synthetic sites, not a province boundary; coordinates are town centroids plus random jitter and do not represent any real service station.

Ranking is not allocation

The investment engine turns the ranking into a capital plan: an exact bounded knapsack over site and intervention pairs under a fixed budget, with a minimum number of funded projects per province.

That provincial constraint exists because a pure return-maximising plan concentrates everything in Gauteng and the Western Cape. That is defensible financially and indefensible as a network strategy — divestment candidates that are the only site within 50 km score badly and should probably still be kept, because the volume does not transfer to a competitor's neighbour 200 km away.

Volume and margin by province. Only South African sites carry a province; every other market is grouped as Outside South Africa, which is the largest row.

PROVINCE	LITRES M	MARGIN M ZAR	MARGIN PCT
Outside South Africa	8.80	17.80	8.86
Gauteng	1.80	4.40	9.80
Western Cape	1.20	2.80	9.67
KwaZulu-Natal	0.60	1.50	10.08
Eastern Cape	0.30	0.70	10.41
North West	0.30	0.70	10.35
Mpumalanga	0.20	0.60	10.64
Free State	0.20	0.60	10.65
Limpopo	0.10	0.30	10.91
Northern Cape	0.10	0.20	10.30

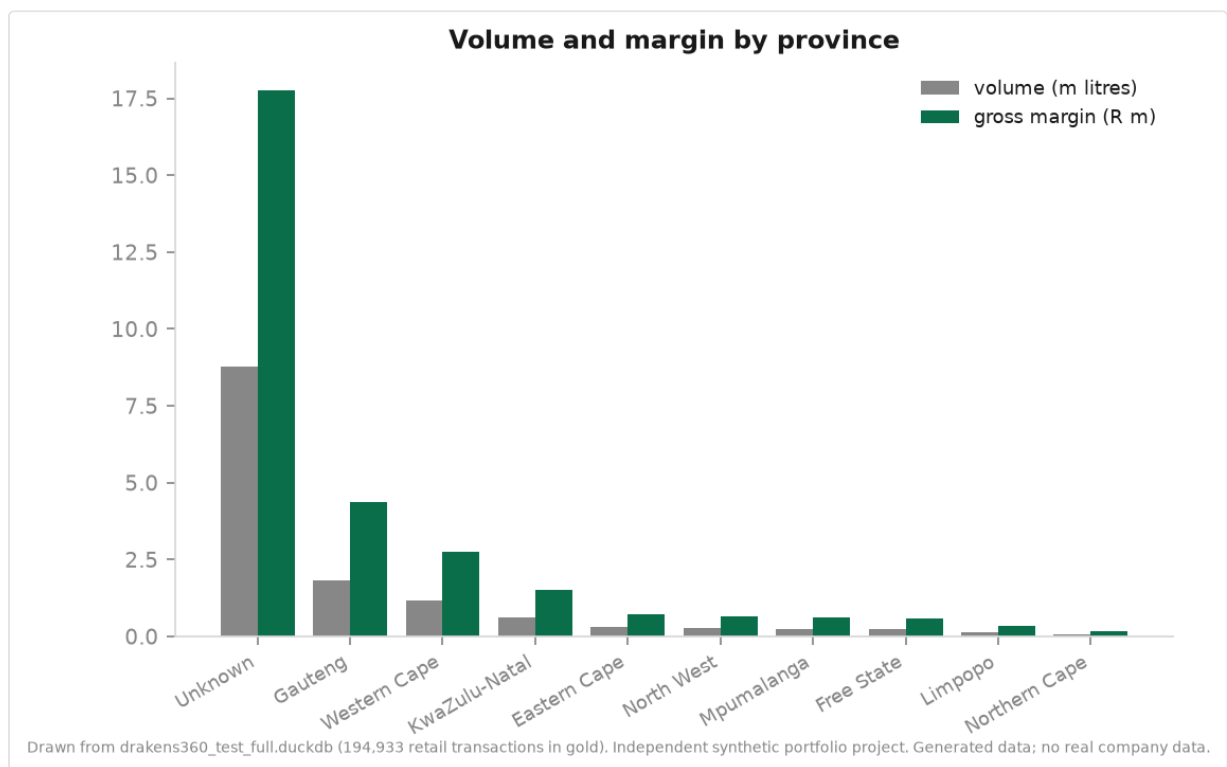


Figure 13. Volume and gross margin by province.

10. Machine learning

Six MLflow experiments: demand forecast, predictive maintenance, customer churn, stock-out risk, late delivery, and unsupervised forecourt anomaly detection. Three rules applied to all of them.

Chronological splits, never random. A random split on time-series data leaks the future into training and produces an accuracy figure that will not survive contact with production.

No feature unknown at prediction time. The stock-out model deliberately excludes `days_of_cover`, even though the target is derived from it — the model would score near-perfectly by reproducing the definition and be worthless the moment the threshold changed.

A baseline that has to be beaten. The demand forecast reports its error against a seasonal-naïve baseline — same weekday, last week — and prints a warning if it fails to beat it. A model that cannot beat “last Tuesday” should not be promoted, however good its R^2 looks.

The anomaly detection is unsupervised because there is no labelled fraud, and in a real downstream business there rarely is. The run is tagged explicitly: a high score means unlike this site's own history, not fraudulent. Most of what surfaces will be a miscalibrated meter or a badly closed shift, and that is still worth finding.

11. Governance

Unity Catalog column masks and row filters, mirrored by Power BI row-level security so a user sees the same rows whichever way they query.

The principle is that the default is restrictive: a new analyst sees masked customer names and no rows outside their region until someone deliberately grants more. Getting this the other way round is how platforms leak.

Two details worth calling out. Analysts get **no bronze access at all** — raw data has no conformed definitions, and any number taken from it will disagree with the dashboard. And the row-filter scope comes from a table rather than hard-coded group names, so adding a region to someone's remit is a data change, not a deployment.

12. Running on Databricks

The full medallion was deployed and executed on a Databricks workspace at portfolio scale. Bronze is generated in-platform rather than uploaded: pushing 37 million rows over a domestic connection takes hours, and Spark produces the same distributions on the cluster in minutes.

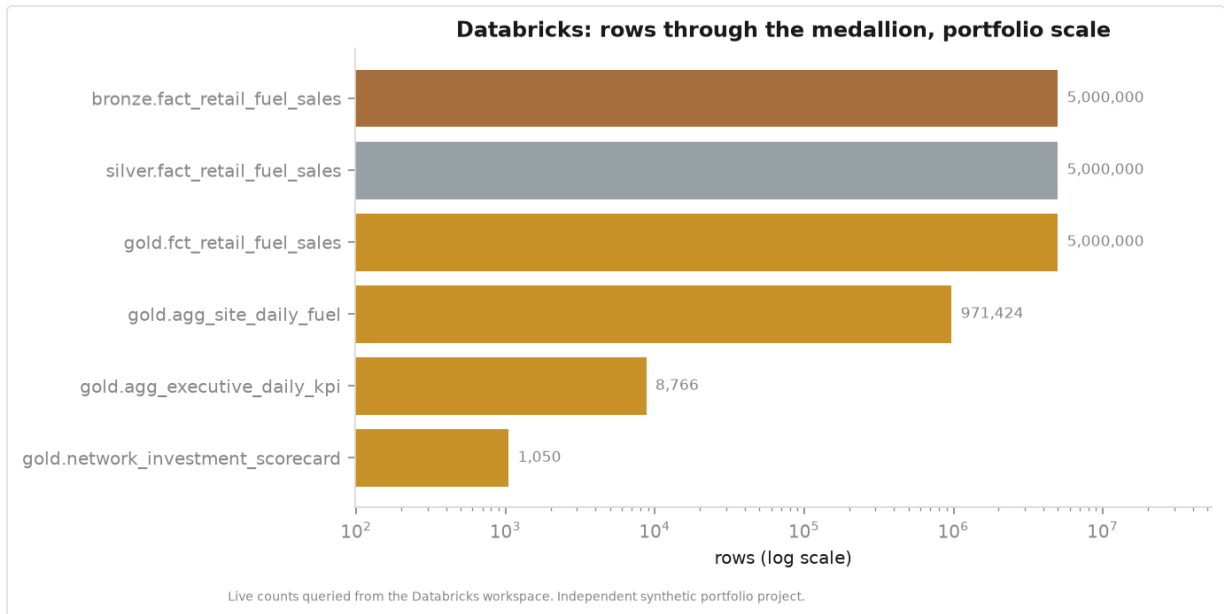


Figure 14. Rows through bronze, silver and gold on Databricks at portfolio scale.

13. What I would do differently

The row-count plan drove the model

Starting from “we need 15 million rows” produced 85 fact tables, some of which exist mainly to carry volume. A real engagement would start from the decisions the business needs to make and let the model follow. The bus matrix would be shorter and better for it.

Uplift assumptions are asserted, not derived

“A convenience build lifts margin 22%” is a planning number I chose. In a real engagement it would come from the sites already converted — which the model can support, but which needs a before/after study rather than an assumption.

Anomaly detection has no feedback loop

The model produces a worklist; a production version would capture investigation outcomes and become supervised within a couple of quarters. Without that it degrades into an alert nobody opens.

Streaming is demonstrated rather than operated

The pipelines, watermarks and windowing are real, but I have not run them under sustained load or handled a poison message mid-stream. I would not claim production streaming experience on the strength of this.

Disclaimer. Independent synthetic portfolio project. This document and the platform it describes contain no data from any real company. Drakens Energy is a fictional company. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident is randomly generated. Company and person names are invented; any resemblance to a real organisation or individual is coincidental and unintended. Site coordinates are town centroids plus random jitter and do not represent any real service station.

Drakens Energy 360 · independent synthetic portfolio project · generated 2026-09-09 from the build manifest, the warehouse and live workspace queries. No figure in this document was typed by hand.